

End-to-End Test-Time Training for Long Context

Motivation. Transformers with full self-attention achieve strong long-context performance but have $O(T^2)$ prefill cost and $O(T)$ per-token decode cost, which becomes prohibitive for very long sequences. RNN-style alternatives (Mamba, DeltaNet) offer constant cost per token but empirically degrade as context grows. Sliding-window attention (SWA) provides a middle ground but cannot access information beyond the window.

The key insight of this paper is that long-context language modeling can be framed as a *continual learning* problem rather than an architecture design problem.

Core Idea: TTT via Next-Token Prediction. Instead of designing new architectures to store long-range context, the authors propose continuing to *train* a standard Transformer (with SWA) at test time on the context itself, using the same next-token prediction loss used during pretraining. The model compresses context into its MLP weights through gradient descent:

$$W_t = W_{t-1} - \eta \nabla \ell_t(W_{t-1}), \quad \text{where} \quad \ell_t(W) = \text{CE}(f(x_{t-1}; W), x_t).$$

After processing T tokens of context, the updated weights W_T encode information beyond the sliding window, enabling the model to attend to the full context implicitly.

End-to-End (E2E) Training. Prior TTT methods (e.g., TTT-KVB) use layer-wise reconstruction losses at training time but next-token prediction at test time, creating a mismatch. TTT-E2E resolves this by making the method end-to-end in two senses:

- **E2E at test time:** The inner loop directly optimizes the next-token prediction loss (not a layer-wise proxy).
- **E2E at training time:** The outer loop optimizes W_0 via meta-learning so that the *post-TTT* loss is minimized, using gradients of gradients through the inner loop.

The training-time objective is:

$$\mathcal{L}(W_0; X) = \frac{1}{T} \sum_{t=1}^T \ell_t(W_{t-1}),$$

where each W_t depends on W_0 through the chain of gradient updates.

Mini-Batch TTT and Sliding Window. Processing tokens one at a time is sequential and unstable. TTT-E2E groups tokens into mini-batches of size b , taking one gradient step per batch. This enables parallelism within each batch and improves stability. The model uses SWA layers for local context (window size k) and TTT for beyond-window context, with $k \geq b$ so the window covers at least one full TTT batch.

Implementation Details. Three design choices are critical:

- (a) **TTT only the MLP layers:** Updating attention or normalization layers in the inner loop causes instability in the outer loop.
- (b) **TTT only the last $L/4$ blocks:** Updating fewer layers reduces compute while maintaining a large effective hidden state. The last layers are chosen as they benefit most from context compression.
- (c) **Two MLPs per block:** In TTT-updated blocks, a second frozen MLP preserves pretrained knowledge, preventing catastrophic forgetting of the base model’s capabilities during test-time adaptation.

Connection to Prior TTT Work. The paper provides an alternative derivation starting from TTT-KVB (Key-Value Binding). TTT-KVB uses per-layer reconstruction losses and separate key/value projections (θ_K, θ_V). TTT-E2E simplifies this by: (1) removing the separate output rule, (2) replacing layer-wise losses with the global next-token prediction loss, and (3) updating fewer but larger MLP layers. This yields a $5\times$ larger effective hidden state with half the latency.

Scaling Results. For 3B-parameter models trained on 164B tokens:

- TTT-E2E scales with context length comparably to full attention — both maintain or improve loss from 8K to 128K tokens.
- Mamba 2, Gated DeltaNet, and TTT-KVB all degrade relative to full attention as context grows beyond 32K.
- TTT-E2E has constant inference latency regardless of context length (like RNNs/SWA), making it $2.7\times$ faster than full attention at 128K context.

Results hold across model sizes from 125M to 3B, with consistent improvements from TTT-E2E on top of SWA even when SWA already has access to full attention at 8K context.

Limitations. TTT-E2E requires backpropagation through the inner-loop gradient steps during training, increasing training cost. The method currently updates only MLP layers in the last quarter of the network, leaving open whether more aggressive updating could help at even longer contexts. The meta-learning outer loop adds complexity compared to standard pretraining.

Takeaway. TTT-E2E reframes long-context modeling as continual learning: rather than building architectures with perfect recall, compress the context into model weights via test-time gradient descent. By making both the inner and outer loops end-to-end with next-token prediction, and leveraging meta-learning to prepare the initialization, TTT-E2E matches full-attention quality at RNN-like inference cost.